1)Credit Scoring Model

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Sample dataset
data = {
   "income": [50000, 20000, 30000, 80000, 100000],
   "loan": [10000, 15000, 12000, 20000, 25000],
   "credit_score": [700, 500, 650, 800, 750],
   "approved": [1, 0, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[["income", "loan", "credit_score"]]
y = df["approved"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2
)

model = LogisticRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Predictions:", predictions)

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

2)Emotion Recognition From Speech

```
import librosa
import soundfile
import numpy as np

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import
train_test_split
from sklearn.metrics import accuracy_score

# Extract audio features
def extract_feature(file_name):

    with soundfile.SoundFile(file_name) as sound:

        audio = sound.read(dtype="float32")

        sample_rate = sound.samplerate

        mfcc = np.mean(
            librosa.feature.mfcc(
                y=audio,
                sr=sample_rate,
                n_mfcc=40
            ).T,
            axis=0
        )

    return mfcc
```

3)Handwritten Character Recognition

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Build model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(X_train, y_train, epochs=3)

# Evaluate
loss, accuracy = model.evaluate(X_test, y_test)

print("Accuracy:", accuracy)
```

4)Disease Prediction Model

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Sample data
data = {
   "age": [25, 40, 35, 50, 60],
   "bp": [120, 140, 130, 150, 160],
   "sugar": [80, 120, 100, 140, 150],
   "disease": [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

X = df[["age", "bp", "sugar"]]
y = df["disease"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2
)

model = RandomForestClassifier()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```